Phonetic name search across Hebrew and Spanish spellings: a review of methods and implementations

Literature review · 10 October 2026

SUMMARY

The archive must return a person's record whether a visitor types the name as it appears in Hebrew script on the stone, in its Argentine Spanish spelling, or in one of the German, Polish or English forms that relatives abroad use. This review surveys five families of methods that address the problem (phonetic keys, phonetically informed string similarity, transliteration-mediated matching, learned cross-script embeddings and name-equivalence dictionaries), describes how each works, and lists the open-source implementations that fit the project's PostgreSQL and Flask stack. A small illustrative probe on 40 constructed name variants (§5) shows the pattern the literature predicts: classical keys are blind to Hebrew script, the Beider–Morse system and romanise-then-encode pipelines bridge the scripts at different costs in precision, and a learned byte-level encoder bridges them best but scores unrelated names as moderately similar. The review closes with the candidate set it implies for the evaluation step (§6).

1 The search problem

Two spellings can denote the same name through three distinct relations, summarised in Figure 1. **Orthographic variation** arises within a script: Yiddish and Hebrew names reached Entre Ríos through German, Polish and Russian spelling conventions and were then respelt with Spanish ones, in which ⟨j⟩ represents /x/, ⟨h⟩ is silent and ⟨b⟩ and ⟨v⟩ are pronounced identically [31], so that one surname surfaces as Schwartz, Szwarc or Svarc and one given name as Chaim, Haim or Jaim. **Cross-script correspondence** links the Hebrew-script inscription to these Latin forms. Hebrew is normally written without vowel points, ⟨ו⟩ may stand for /v/, /o/ or /u/, and the letters ⟨ב⟩, ⟨כ⟩ and ⟨פ⟩ alternate between stop and fricative values that unpointed text leaves unmarked [21, 22]; a single Hebrew spelling therefore corresponds to many Latin readings, and a single Latin spelling to several Hebrew ones. **Onomastic equivalence** links names related by translation or by naming custom, such as Yitzchak and Isaac, or the Hebrew name Rachel carried by a woman registered as Cecilia, the example raised in the team's schema discussion. Phonetic resemblance plays no part in the third relation, which the genealogical literature handles with curated name dictionaries [23]. The Fuzzy Name Search feature in the project specification targets the first two relations, and the review concentrates on them while returning to the third in §6.

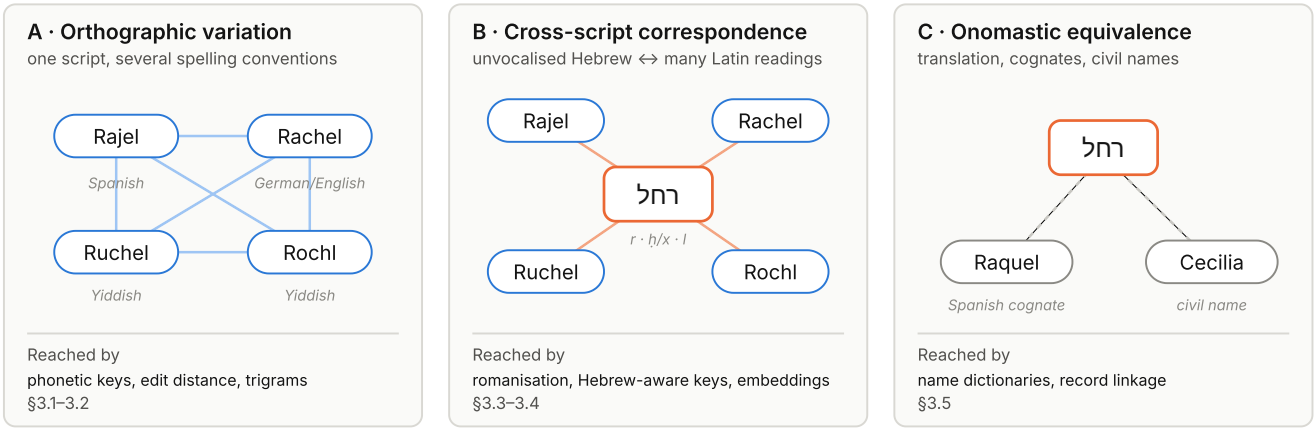


Figure 1. Three relations under which two spellings denote one name, illustrated with Rachel. Search methods differ chiefly in which relations they can reach; only panels A and B involve sound, and panel C lies outside the reach of any phonetic method. The Cecilia–Rachel pairing is the example recorded in the team's schema discussion of 30 September.

2 Map of the literature

Name matching sits at the junction of genealogy, record linkage and multilingual information retrieval, and the three communities have worked largely in parallel (Figure 2). Genealogists produced the rule-based phonetic keys, culminating in two systems designed for Ashkenazi names, Daitch–Mokotoff Soundex and Beider–Morse Phonetic Matching [2, 5]. Record-linkage research compared those keys against string-similarity measures and found no

technique that dominates across datasets [11]. Work on Arabic, the closest Semitic analogue to the present problem, showed that English-centred keys perform poorly on abjad scripts and that encoding one-to-many letter correspondences recovers much of the loss [12, 13]. Multilingual retrieval moved the problem into learned vector spaces, first with linear projections and more recently with neural encoders trained on millions of cross-script name pairs [16, 19, 20].

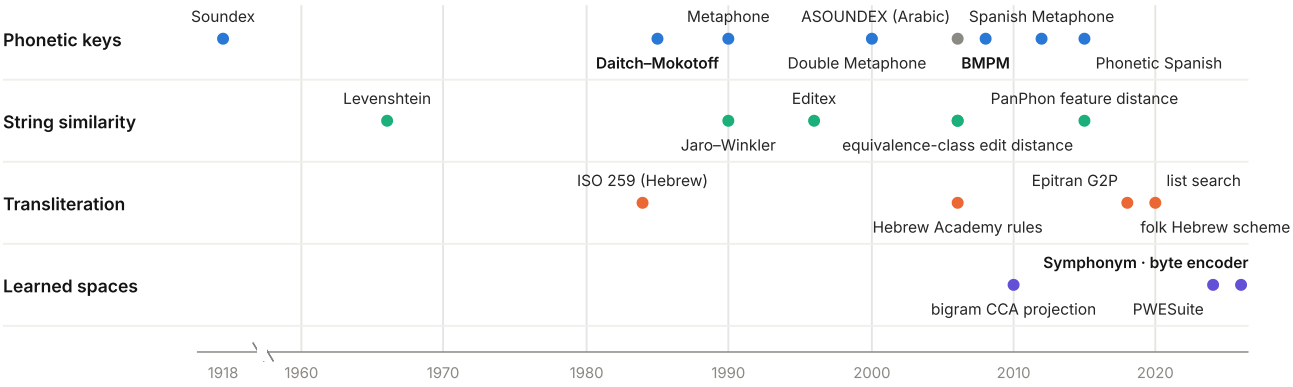


Figure 2. Principal methods by family and year of publication, drawn from the sources in the reference list (the ASOUNDEX point, shown in grey, adapts Soundex to Arabic script). Bold labels mark the two systems built for Ashkenazi Jewish names.

3 Methods

3.1 Phonetic keys

A phonetic key maps a name to a short code such that names that sound alike receive the same code; search then reduces to an equality or overlap test that a database index answers cheaply. **Soundex**, patented by Russell in 1918, keeps the initial letter and encodes up to three following consonants as digits in six classes, discarding vowels [1]. Its English-centred classes and its dependence on the first letter make it fragile for Yiddish names: Chaim and Haim receive C500 and H500. **Metaphone** replaces digit classes with context-sensitive spelling rules for English, and **Double Metaphone** adds rules for names of other European origin, emitting a primary and an alternate code for ambiguous names [3, 4].

Daitch–Mokotoff (D–M) Soundex, designed in 1985 for Slavic and Yiddish surnames, codes multi-letter sequences such as ⟨sch⟩, ⟨sz⟩ and ⟨tz⟩ as single sounds, extends the code to six digits and, crucially, branches: a sequence with two plausible pronunciations yields two codes, and two names match when their code sets intersect [2]. PostgreSQL has shipped it since version 16 as `daitch_mokotoff()`, which returns a text array that a GIN index searches with the overlap operator `&&` [24]. Two limitations matter here. The code tables were built for German and Slavic orthography, so Spanish ⟨j⟩ is coded as /j/ or /dʒ/ (classes 1 and 4) and never as /x/ (class 5); Jaim matches Chaim only through the alternative ‘tsh’ reading of ⟨ch⟩ and never matches Haim. The function also returns no code for Hebrew-script input (Figure 3).

Spanish-specific keys address the first limitation. **Spanish Metaphone** merges ⟨b/v⟩ and ⟨ll/y⟩, renders ⟨j⟩ and soft ⟨g⟩ alike and drops ⟨h⟩ [6, 28], and **Phonetic Spanish** applies a digit-class scheme in the same spirit; in a record-linkage comparison of Spanish encodings the latter performed best in most conditions [7]. Both assume that every name is Spanish, which the colony’s Yiddish surnames are not.

Beider–Morse Phonetic Matching (BMPM) is the most developed system for this problem [5]. It first estimates the language of the name from about 200 spelling patterns, then rewrites the name into a restricted phonetic alphabet with language-specific rules, producing every plausible pronunciation as a separate token. The exact pass also applies final devoicing and assimilation, an approximate pass merges partially interchangeable sounds such as unstressed vowels, and a further *Hebrew phonetic value* pass reduces each token to a consonant-dominant skeleton so that names written in or transliterated from Hebrew can match. Hebrew script and Spanish are both among its supported languages, and the authors note that Grinberg and Grimberg are exactly equivalent in Argentina; they report substantially fewer false hits than soundex-style codes, at the price of missing some anglicised forms such as Silver for Silber [5]. The widely used ports (Apache Commons Codec, Elasticsearch, Solr, Abydos) implement

language detection and the exact and approximate passes; Commons Codec and Abydos both ship the Hebrew-value rule files, but neither engine applies them [26, 27, 28]. Figure 3 shows the codes each key assigns to two name families.

Input	Soundex	Double Metaphone	Daitch–Mokotoff	BMPM approx. tokens (count)	BMPM Hebrew value
Schwartz family					
שוורץ	—	—	—	sVrC (1)	svrC
Svarc (Sp.)	S162	SFRK	479400 479500	svarts, svark, … (12)	svrC, …
Schwartz	S632	XFRT XRTS	479400	svarts, svorts, … (3)	svrC
Szwarc	S620	SRK XRK	479400 479500	svarts, svorts (2)	svrC
Chaim family, with an unrelated surname					
חיים	—	—	—	Xim (1)	xim
Jaim (Sp.)	J500	AM JM	160000 460000	xim, iam, … (15)	xim, …
Chaim	C500	XM	460000 560000	xim, sam, … (15)	xim, …
Haim	H500	HM	560000	xim, am, … (12)	xim, …
Kohn (unrelated)	K500	KN	560000	kon (1)	kun

Figure 3. Keys assigned by each phonetic system, computed with PostgreSQL 16 `fuzzystrmatch` and Apache Commons Codec. Blue codes are shared within a family, which is what produces a match; orange marks the collision through which D–M equates Chaim and Haim with the unrelated surname Kohn, since ⟨ch⟩/⟨h⟩ and ⟨k⟩ share class 5 and ⟨m⟩ and ⟨n⟩ share class 6. The Hebrew-value column applies BMPM's published Hebrew rule file (step 4 of the algorithm) to both scripts through a small patch to Commons Codec; it is a reconstruction for illustration, since no maintained port exposes this step. The same tokens result from a few lines of configuration in Abydos, which makes the pass available in Python.

3.2 Phonetically informed string similarity

Similarity measures score a pair of strings on a continuum and so support ranked results and thresholds. **Levenshtein** distance counts the edits needed to turn one string into the other [8], and **Jaro–Winkler** rewards shared characters in nearby positions with extra weight on a common prefix [9]; neither knows that ⟨v⟩ and ⟨w⟩ are closer than ⟨v⟩ and ⟨k⟩. **Editex** introduced letter groups into the edit cost, so that substitutions within a phonetic group are cheap, combining the graded scores of edit distance with the sound classes of phonetic keys [10]. Freeman et al. extended the same idea across scripts for Arabic and English, replacing the equality test with sets of acceptable Latin renderings for each Arabic letter (⟨ق⟩ may be q, g or k), retaining only vowels present in both strings, and raised the F-score on news names from about 0.48 to about 0.80 [12]. Their equivalence-class design transfers directly to Hebrew, where ⟨ח⟩ admits ch, kh, h and Spanish j. **PanPhon** generalises the cost further by representing each IPA segment as 22 articulatory features and charging edits by the features that change [14]; it requires an IPA transcription, which the Epitran grapheme-to-phoneme library supplies for Spanish but not, in the current release, for Hebrew [15].

The **trigram similarity** of PostgreSQL's `pg_trgm` extension is the pragmatic member of this family: it compares the sets of three-character substrings of two strings, supports GIN and GiST indexes, and matches above a configurable threshold (0.3 by default) [24]. It is fast, already in the project design and well suited to typing errors, but it carries no phonetic knowledge and cannot relate strings in different scripts.

3.3 Transliteration-mediated matching

The most common engineering answer to cross-script search is to romanise the Hebrew form and then apply a Latin-script method. The choice of romanisation is consequential. Scholarly standards such as ISO 259 and the Academy of the Hebrew Language rules aim at reversibility [22], whereas name search needs output that resembles what people type; Babel Street's commercial name matcher replaced the UNGEGN scheme available through ICU with a deliberately lossy 'folk' scheme for this reason, guessing vowels and the vocal or silent value of shva [21]. Aqeel et al. reached a similar conclusion for Arabic: among the Arabic-tailored techniques they evaluated, which include an Arabic Soundex (ASOUNDEX), the best improved retrieval by about 70 % over existing approaches designed around Latin script [13]. A simple pipeline of the AnyAscii transliterator followed by D–M recovers many Hebrew–Latin pairs in the probe of §5, because D–M discards vowels that the romaniser cannot supply, but it also inherits D–M's collisions (Figure 5). Two refinements recur in the literature: emitting several romanisations per Hebrew name, one per plausible vocalisation, and generating romanisations with a

trained transliteration model, whose quality Khakhmovich et al. recommend judging by downstream list-search accuracy [17]. For this project the ingestion pipeline already calls Gemini on every stone, which offers a third route: asking the model for the Latin readings of each Hebrew name at transcription time.

3.4 Learned cross-script representations

Learned methods map names from every script into one vector space in which distance tracks pronunciation, so that a query in either script is answered by nearest-neighbour search. Udupa and Khapra projected character-bigram features of English and native-script names into a shared 50-dimensional space with canonical correlation analysis, trained on about 15,000 pairs per language, and reported a mean reciprocal rank of 0.72 for Hebrew queries against English Wikipedia titles [16]. PWESuite formalised the evaluation of phonetic word embeddings built from articulatory features [18]. Two 2026 systems are directly relevant. **Symphonym** distils an IPA-feature teacher into an 8.3-million-parameter character-level student that embeds names from twenty writing systems, Hebrew included, in 128 dimensions; on a benchmark of expert-matched medieval Hebrew and Arabic toponyms it reached 85.2 % recall at rank 1, against 81.5 % for Levenshtein distance after AnyAscii romanisation, a margin that shows how strong the romanise-then-compare baseline already is [19]. A byte-level contrastive encoder from TU Delft (six layers, about four million parameters, no tokeniser) was trained on 4.67 million person-name pairs generated with LLMs from Wikidata across eight scripts including Hebrew; it reports recall at rank 10 above 0.95 for Hebrew and an MRR of 0.78 overall, against about 0.09 for Levenshtein, Double Metaphone and BM25 on its predominantly cross-script queries [20]. Its checkpoint is public and small enough to run in a Flask process on CPU (about 13 ms per name in the unoptimised NumPy port written for this review), and it is the encoder used in §5.

Two cautions follow from the same sources. Both systems were trained on synthetic or gazetteer pairs, and neither reports results on Yiddish names in Spanish orthography. General-purpose sentence-embedding models, considered in the team's 25 September meeting, are trained for semantic similarity; none of the surveyed papers evaluates them on cross-script name matching, so their suitability is an empirical question for the next step.

3.5 Name-equivalence dictionaries

Onomastic equivalence requires knowledge that pronunciation does not encode. The JewishGen Given Names Data Bases, compiled by Esterson and Curwin, link each 'legal' Hebrew or Yiddish name to the Yiddish and vernacular names associated with it in fifteen European regions and ten destination countries, Argentina among them; the destination links were built partly from cemetery records [23]. A table of such pairs supports query expansion: a search for Isaac is rewritten to include Yitzchak and יצחק before any phonetic method runs. The archive will itself generate equivalence evidence, since each transcribed person may carry a Spanish and a Hebrew name on the same stone.

4 Implementations

Table 1 lists the implementations that fit the project stack. PostgreSQL covers phonetic keys, trigrams and vector search natively; BMPM and transliteration run in Python or Java at ingestion and query time, with their tokens stored in indexed array columns.

Implementation	Methods	Hebrew input	Spanish rules	Fit for this project
PostgreSQL `fuzzystrmatch` [24]	Soundex, Metaphone, Double Metaphone, D–M, Levenshtein	no	no	Native; D–M array + GIN &&; the Soundex and Metaphone functions are documented as unreliable on UTF-8 text
PostgreSQL `pg_trgm` [24]	Trigram similarity, word similarity, KNN distance	within script	n/a	Native; already planned for normalised names
`pgvector` [25]	Vector storage, cosine distance <=>, HNSW and IVFFlat indexes	via encoder	via encoder	Native extension; holds learned embeddings
Elasticsearch phonetic plugin [26]	BMPM (`languageset` may include hebrew, spanish), D–M, Metaphone	BMPM steps 1–3	yes	Reference configuration; a separate service is heavier than the stack needs
Apache Commons Codec [27]	BMPM (generic, Ashkenazi, Sephardic; exact, approx.)	steps 1–3	yes	Java; reference port used in this review

Implementation	Methods	Hebrew input	Spanish rules	Fit for this project
Abydos [28]	BMPM, D–M, Spanish Metaphone, Phonetic Spanish, Editex and many more	steps 1–3	yes	Python; runs inside Flask and `import.py`; Hebrew-value pass reachable by configuration
Epitran + PanPhon [14, 15]	Grapheme-to-IPA, articulatory feature distances	no map	yes	Useful for Spanish-side feature distances only
AnyAscii, ICU transforms [29]	Rule-based romanisation	yes	n/a	Cheap first stage for romanise-then-encode
Byte-level encoder [20], Symphonym [19]	Cross-script name embeddings	yes	untested	Public weights; CPU inference (under 1 ms per name reported for Symphonym)
Babel Street Match [21]	Commercial cross-script name matching	yes	yes	Licensed product; a benchmark of practice, outside the budget

Table 1. Implementations surveyed. 'Steps 1–3' means that language detection and the exact and approximate passes are implemented and the Hebrew-value pass is not, so Hebrew-script tokens do not intersect Latin-script ones without further work.

5 An illustrative probe

> **Scope of this section.** The 40 names below are constructed variants chosen to exercise the phenomena in §1 (eight families of five spellings, each with one Hebrew-script form and at least one Spanish-orthography form). They are not drawn from the transcription CSV and the figures are not an evaluation; the evaluation is the next step of the task. Thresholds: trigram 0.3 (PostgreSQL default), embedding cosine 0.75 (chosen for this illustration).

Each method was run on all 780 pairs of the 40 names. Figure 4 shows the resulting match matrices, in which a perfect method would fill the eight diagonal blocks and nothing else, and Figure 5 summarises them. Soundex, Double Metaphone and trigrams recover only part of the Latin-script variation and none of the Hebrew forms. D–M recovers most Latin variation but merges the Chaim and Cohen families. BMPM without its Hebrew pass recovers somewhat less Latin variation than D–M with no false matches; adding the Hebrew-value pass bridges half of the Hebrew–Latin pairs, including the decisive Spanish ones (Jaim, Svarc, Rajel, Rebeca, Grinberg), still without false matches, but misses names whose Latin spelling carries a vowel that the Hebrew omits (Kohan, Rivka). AnyAscii followed by D–M bridges more pairs and produces the most false matches. The byte-level encoder bridges the most pairs with a single false match, but its continuous panel shows a high floor of similarity between unrelated names (mean cosine 0.34 between names of different families, maximum 0.76, with 54 of the 700 such pairs above 0.6), which in an archive of about a thousand people would push spurious candidates into the results unless the score is combined with other evidence.

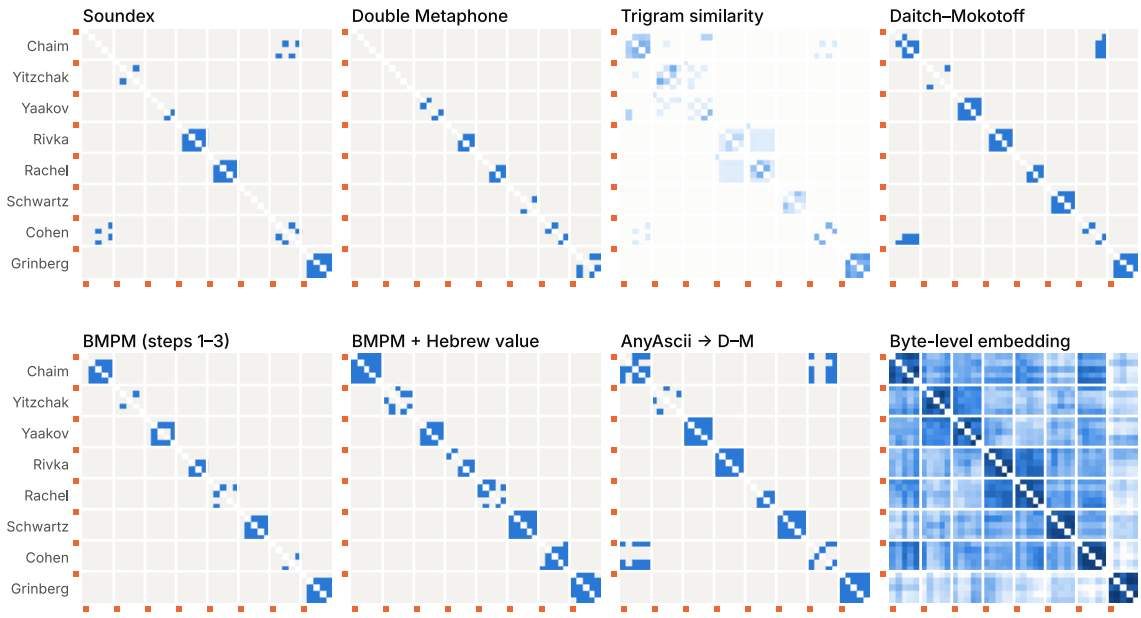


Figure 4. Pairwise match matrices for the 40-name probe, ordered by family; white rules separate the eight families, and orange marks on the axes indicate the Hebrew-script entry of each family. Blue cells are matches (binary keys) or cosine and trigram similarity on a sequential scale from 0 to 1 (trigram and embedding panels).

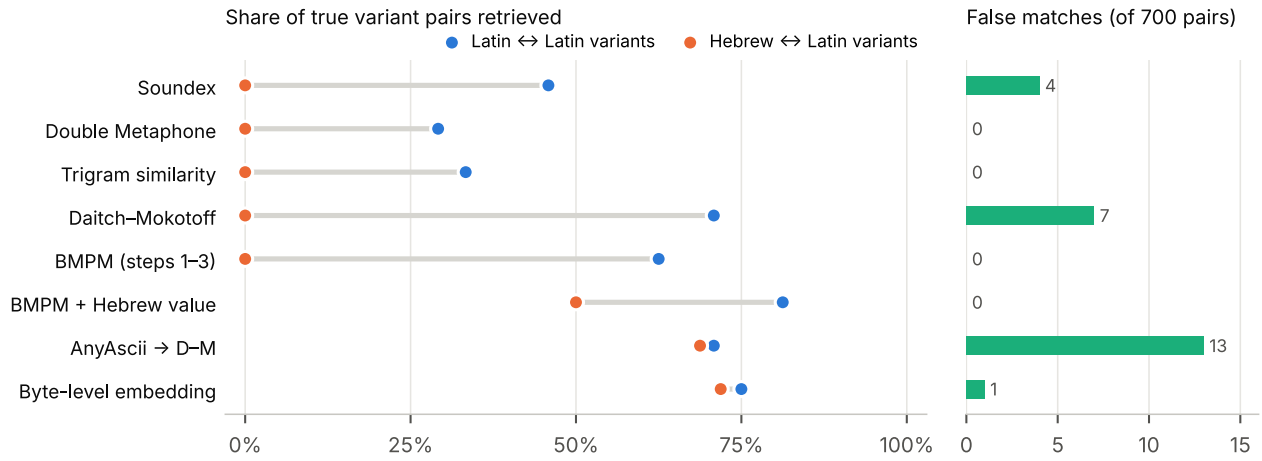


Figure 5. Share of true variant pairs retrieved, separately for pairs within Latin script (48 pairs) and pairs across scripts (32 pairs), and the number of false matches among the 700 pairs from different families.

Onomastic pairs behave as §1 anticipates (Table 2). The phonetic systems match only those cognates that happen to sound alike (Rachel–Raquel, and Moshe–Moisés under BMPM's approximate rules) and occasionally by coincidence (Chaim–Jaime under D–M). The encoder assigns these pairs cosines of 0.43–0.92, a range that overlaps the 0.76 reached by unrelated names, so the score cannot by itself separate genuine equivalents from coincidental resemblance.

Pair	Relation	D–M	BMPM + Hebrew value	AnyAscii → D–M	Embedding cosine
Rachel – Raquel	Spanish cognate	match	match	match	0.92
Moshe – Moisés	Spanish cognate	—	match	—	0.89
משה – Moisés	cognate, cross-script	—	—	—	0.70
יצחק – Isaac	cognate, cross-script	—	—	—	0.62
Chaim – Jaime	phonetically close Spanish name	match	—	match	0.68
רחל – Cecilia	civil name	—	—	—	0.43

Table 2. Onomastic pairs from the probe. Orange marks matches that arise from code collisions. Jaime is included as a Spanish name phonetically close to Chaim; whether the two denote one person is a question for a name dictionary and the records.

6 Implications for the candidate set

The literature and the probe agree that no single method covers both relations at acceptable precision, in line with Christen's finding that no technique dominates across monolingual name sets [11]. A design that runs several candidate generators and merges their outputs follows naturally, and Figure 6 sketches one consistent with the archive's data tier. The derived columns are computed once in `import.py`, in line with the existing design for normalised names and D–M codes, and each generator is an indexed query, so a search costs a handful of index scans over about a thousand people. Candidate lists can be merged without score calibration by reciprocal rank fusion, which sums $1/(k+\text{rank})$ across lists [30], and a name-equivalence table supplies expansions before retrieval.

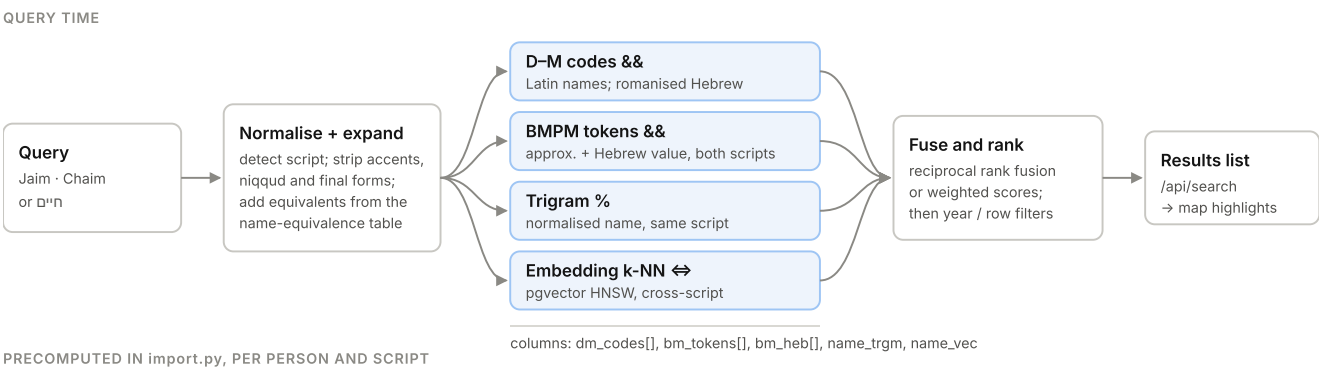


Figure 6. A hybrid design consistent with the literature and with the existing data-tier plan. Generators in blue are independent indexed lookups; each can be switched off, which turns the evaluation step into an ablation over the same pipeline.

For the evaluation step, the review suggests the following candidate set and protocol.

- **Candidates.** (i) D–M on Latin names, the current design and the baseline; (ii) BMPM with the Hebrew-value pass (Abydos, configured as in Figure 3), configured with a fixed language set (Spanish, Hebrew, German, Polish, Russian), since automatic detection classes Svarc as Hungarian, Czech, Romanian or Turkish; (iii) romanise-then-encode, with AnyAscii and with Gemini-generated readings; (iv) the byte-level encoder and Symphonym through pgvector; (v) the fusion of the best performers, with and without the equivalence table.
- **Gold standard.** Every transcribed person whose stone carries both a Latin and a Hebrew name provides a positive cross-script pair; pairs that are onomastic equivalents should be labelled separately, since they test the equivalence table. Latin-script variants for queries can be drawn from typical diaspora spellings and from deliberate typing errors.
- **Metrics.** Recall at 10 and mean reciprocal rank, the measures used by the cross-script studies above [16, 19, 20], reported separately for Latin-to-Hebrew, Hebrew-to-Latin and Latin-to-Latin queries, together with the number of unrelated records returned in the first ten results.

References

[1] Russell, R. C. (1918). *Index*. US Patent 1,261,167.

[2] Mokotoff, G. Soundexing and genealogy. Avotaynu. https://www.avotaynu.com/soundex.htm

[3] Philips, L. (1990). Hanging on the Metaphone. *Computer Language*, 7(12).

[4] Philips, L. (2000). The Double Metaphone search algorithm. *C/C++ Users Journal*, 18(6).

[5] Beider, A., & Morse, S. P. (2008). Beider–Morse Phonetic Matching: An alternative to Soundex with fewer false hits. *Avotaynu*. https://stevemorse.org/phonetics/bmpm.htm

[6] Mosquera, A. Spanish Metaphone. https://github.com/amsqr/Spanish-Metaphone

[7] Angeles, M. del P., & Bailón-Miguel, N. (2016). A comparative of Spanish encoding functions: Efectiveness on record linkage. *Proc. ICTRS 2016*, 105–113. doi:10.5220/0006227701050113

[8] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10(8), 707–710.

[9] Winkler, W. E. (1990). String comparator metrics and enhanced decision rules in the Fellegi–Sunter model of record linkage. *Proc. Section on Survey Research Methods, American Statistical Association*, 354–359.

[10] Zobel, J., & Dart, P. (1996). Phonetic string matching: Lessons from information retrieval. *Proc. SIGIR '96*, 166–172.

[11] Christen, P. (2006). A comparison of personal name matching: Techniques and practical issues. *Proc. ICDM Workshops*, 290–294. doi:10.1109/ICDMW.2006.2

[12] Freeman, A., Condon, S., & Ackerman, C. (2006). Cross linguistic name matching in English and Arabic. *Proc. HLT-NAACL*, 471–478.

[13] Aqeel, S. U., Beitzel, S., Jensen, E., Grossman, D., & Frieder, O. (2006). On the development of name search techniques for Arabic. *JASIST*, 57(6), 728–739. doi:10.1002/asi.20323

[14] Mortensen, D. R., Littell, P., Bharadwaj, A., Goyal, K., Dyer, C., & Levin, L. (2016). PanPhon: A resource for mapping IPA segments to articulatory feature vectors. *Proc. COLING 2016*, 3475–3484.

[15] Mortensen, D. R., Dalmia, S., & Littell, P. (2018). Epitran: Precision G2P for many languages. *Proc. LREC 2018*. (Hebrew map absent in v1.35.3, checked October 2026.)

[16] Udupa, R., & Khapra, M. M. (2010). Improving the multilingual user experience of Wikipedia using cross-language name search. *Proc. NAACL-HLT*, 492–500.

[17] Khakhmovich, A., Pavlova, S., Kirillova, K., Arefyev, N., & Savilova, E. (2020). Cross-lingual named entity list search via transliteration. *Proc. LREC 2020*, 4247–4255.

[18] Zouhar, V., Chang, K., Cui, C., Carlson, N. B., Robinson, N. R., Sachan, M., & Mortensen, D. R. (2024). PWESuite: Phonetic word embeddings and tasks they facilitate. *Proc. LREC-COLING 2024*, 13344–13355.

[19] Gadd, S. (2026). Symphonym: Universal phonetic embeddings for cross-script name matching. arXiv:2601.06932v4. Weights: https://huggingface.co/docuracy/symphonym-v7

[20] Jumle, V., et al. (2026). Cross-script phonetic name retrieval via byte-level contrastive embeddings. TU Delft DSAIT4050 project. https://github.com/vedant-jumle/cross-language-phonetic-text-alignment; summary in *Towards Data Science*, "Bytes speak all languages".

[21] Babel Street (2020). Building a more useful Hebrew transliteration scheme. https://www.babelstreet.com/blog/building-a-more-useful-hebrew-transliteration-scheme

[22] Romanization of Hebrew. *Wikipedia* (accessed 10 October 2026), for the standards and the ambiguities of unpointed text.

[23] Esterson, G. L., & Curwin, D. Given Names Data Bases. JewishGen. https://www.jewishgen.org/databases/givennames/

[24] PostgreSQL Global Development Group. *fuzzystrmatch* and *pg_trgm* documentation (v18); Release notes 16 (2023-09-14). https://www.postgresql.org/docs/current/

[25] pgvector: open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector

[26] Elastic. Phonetic token filter (analysis-phonetic plugin). https://www.elastic.co/guide/en/elasticsearch/plugins/current/analysis-phonetic-token-filter.html

[27] Apache Software Foundation. Commons Codec, `org.apache.commons.codec.language.bm`. https://github.com/apache/commons-codec

[28] Little, C. C. Abydos: NLP/IR library for Python. https://abydos.readthedocs.io

[29] Hunter WB. AnyAscii: Unicode to ASCII transliteration. https://github.com/anyascii/anyascii

[30] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *Proc. SIGIR '09*, 758–759.

[31] Spanish orthography. *Wikipedia* (accessed 10 October 2026).

Reproducibility: the probe ran PostgreSQL 16 `fuzzystrmatch`/`pg_trgm`, Apache Commons Codec (main branch, October 2026) with a patch applying `gen_hebrew_common.txt`, AnyAscii, and a NumPy port of the byte-level encoder checkpoint `best_v2`; scripts accompany this PDF.